Gestion des exceptions

Dany Siriphol – CODA 1 ère année - 2025

Exceptions

Une exception est un événement qui interrompt le flux d'exécution d'un programme lorsqu'une erreur se produit

Exemples :

- StackOverflowError
- NullPointerException
- IOException

try catch

Syntaxe

```
try {
    // code à surveiller
} catch ( Exception e ) {
    // gestion d'erreur
}
```

try catch

Syntaxe

Le bloc finally est un bloc qui exécute dans tout les cas de figure

```
try {
    // code à surveiller
} catch ( Exception e ) {
    // gestion d'erreur
} finally {
    // code executé dans tout les cas
}
```

Exemple

```java
String text = null;

try {
    System.out.println(text.length());
} catch (NullPointerException e) {
    System.out.println("Erreur : objet null");
}
```

Exceptions

ArithmeticError	Occurs when a numeric calculation goes wrong
ArrayIndexOutOfBoundsException	Occurs when trying to access an index number that does not exist in an array
ClassFormatError	Occurs when a class file cannot be accessed
ClassNotFoundException	Occurs when trying to access a class that does not exist
ConcurrentModificationException	Occurs when an element is added or removed from iterables
FileNotFoundException	Occurs when a file cannot be accessed
IncompatibleClassChangeError	Occurs when there's been a change in a base class after a child class has already been initialized
InputMismatchException	Occurs when entering wrong input (e.g. text in a numerical input)
InterruptedException	Occurs when a Thread is interrupted while waiting/sleeping
InvalidClassException	Occurs when the Serialization runtime observes a problem with a class
IOException	Occurs when an input or output operation fails
NegativeArraySizeException	Occurs when trying to create an array with negative size
NoClassDefFoundError	Occurs when the class is not found at runtime
NoSuchFieldException	Occurs when trying to access a class field/variable that does not exist

Lever des exceptions

Syntaxe

Il est possible de lever « manuellement » des Exceptions avec le mot clé **throw**

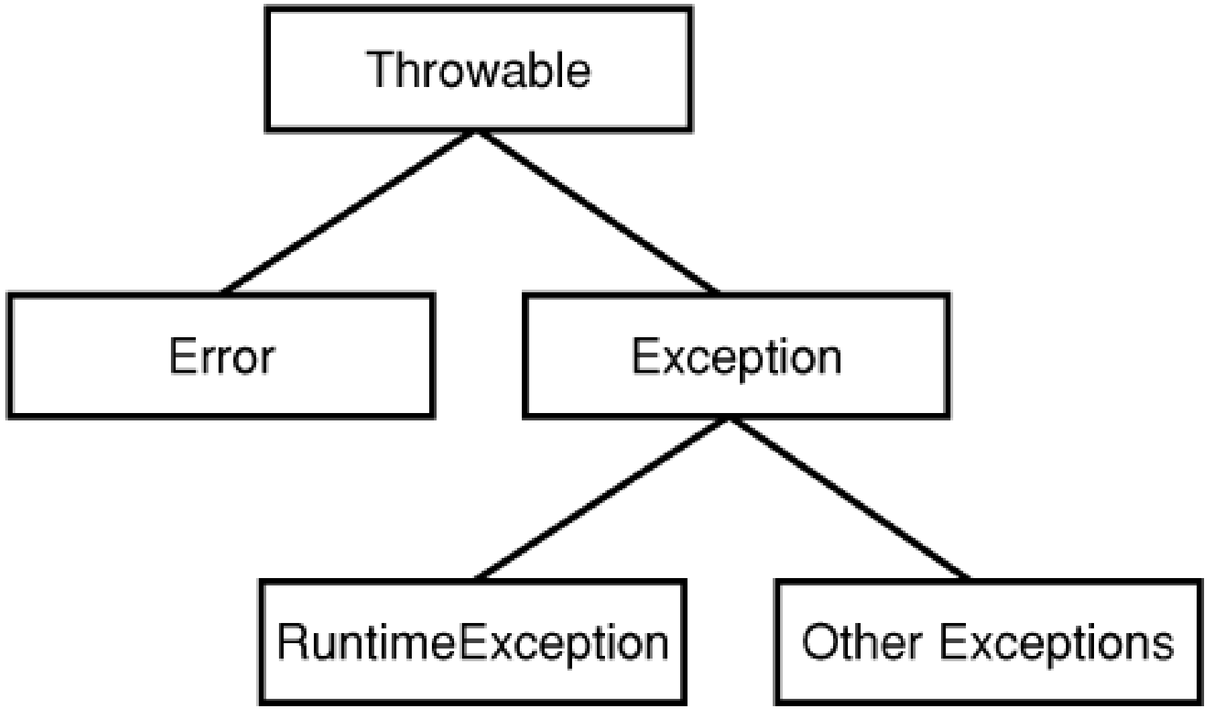
```
Scanner in = new Scanner(System.in);
int age = in.nextInt();

try {
    if ( age < 0 )
        throw new Exception("Pas d'age négatif");

    Person p = new Person();
    p.age = age;
} catch ( Exception e ) {
    System.out.println(e);
}
```

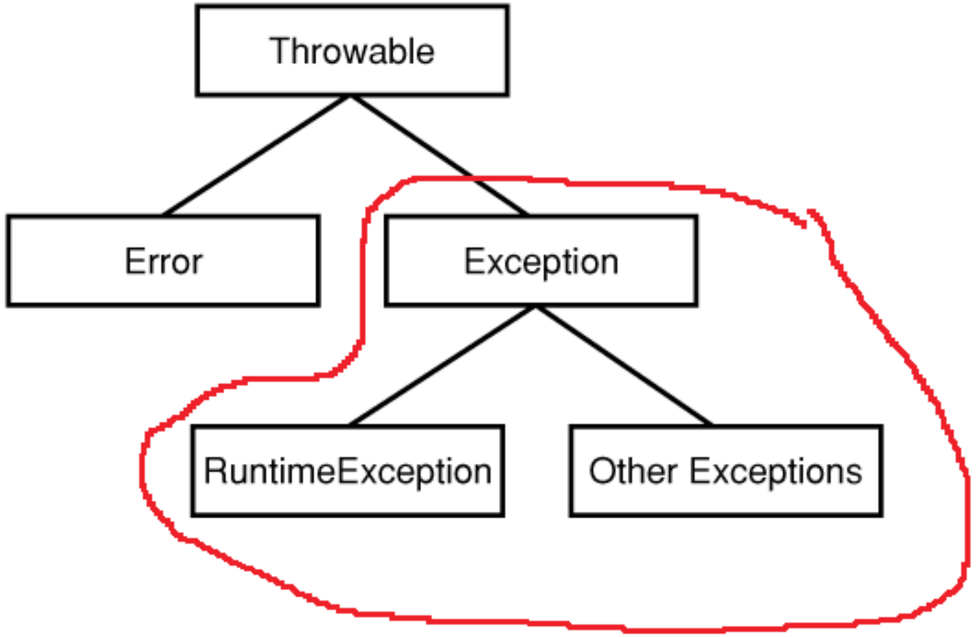
Types d’exceptions

Hiérarchie des exceptions



Exceptions non contrôlées

Ce sont des exceptions « implicites » qui se déclenchent pendant l’execution du programme. Elles correspondent à des erreurs générales :
Division par 0, référence null, erreur d’index de tableau



Exceptions contrôlées

Ce sont des exceptions «explicites » qui sont détectées lors de la compilation. Elles sont définies par le programmeur et lancées explicitement dans le code. Ces exceptions peuvent être natives à Java ou crée pour le projet

- IOException : Erreur d'entrée/sortie (ex: lecture/écriture d’un fichier)
- SQLException : Erreur liée à une base de données
- FileNotFoundException : Fichier introuvable lors de l’ouverture

Capturer plusieurs types d'Exception

Syntaxe

```
try {
    // ...
} catch ( ArithmeticException e) {
    // erreur arithmetique
} catch ( Exception e ) {
    // erreur generale
}
```

Plusieurs captures dans un même catch

Syntaxe

```
try {
    // ...
} catch (IllegalArgumentException | SecurityException e) {
    // ...
} catch ( Exception e ) {
    // erreur generale
}
```

Exception et méthodes

Exceptions et méthodes

Il est possible d’utiliser des try catch dans des méthodes.

```
public Person(String firstname, String lastname, int age){

    try {
        if ( this.age < 0 )
            throw new Exception("age négatif");
    } catch ( Exception e ) {
        this.age = 0;
    }

    this.firstname = firstname;
    this.lastname = lastname;
}
```

Exceptions et méthodes

Une méthode qui lève une exception peut ne pas capturer l'exception et déléguer son traitement à la méthode appelante.

Toute méthode qui délègue une exception doit le signaler dans sa signature avec le mot clé **throws**

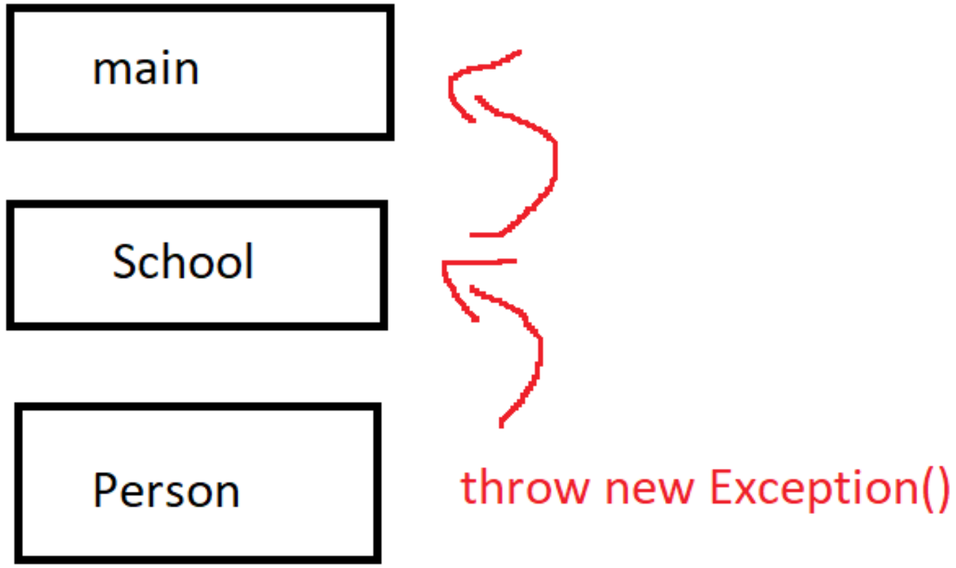
```
public Person(String firstname, String lastname, int age) throws Exception {

    if ( this.age < 0 )
        throw new Exception("age négatif");

    this.firstname = firstname;
    this.lastname = lastname;
    this.age = age;
}
```

Délégation d’exception

Les exceptions qui sont levé mais non capturé dans une méthode remontent dans la chaîne d’appel du programme jusqu’au prochain try catch



Délégation d’exception

```java
public Person(String firstname, String lastname, int age){

    try {
        if ( this.age < 0 )
            throw new Exception("age négatif");
    } catch ( Exception e ) {
        this.age = 0;
    }

    this.firstname = firstname;
    this.lastname = lastname;
}
```

```java
Person p;

try {
    p = new Person("John", "Doe", -5);
} catch ( Exception e ) {

}
```

Exceptions sur méthodes natives Java

Remarque : Certaines méthodes d’objet natif de Java lèvent naturellement des exceptions. Ces méthodes doivent être obligatoirement dans des try catch

```
Connection connection = DriverManager.getConnection(...);
```

```
URL url = new URL("https://www.google.com");
InputStream stream = url.openStream();
```

Créer ses propres exceptions

Créer un nouveau type Exception

Syntaxe

Il est possible de créer ses propres classes d’Exception pour gérer des situations particulières

Il faut pour cela créer une nouvelle classe qui hérite d’Exception

```
public class CustomException extends Exception {

}
```

Créer un nouveau type Exception

Syntaxe

La nouvelle classe d'Exception possède son propre constructeur et ses propres méthodes afin de lui attacher son propre comportement

```
public class WrongCardException extends Exception {

    private Card card;

    public WrongCardException(Card c) {
        this.card = c;
    }

    public String toString() {
        return "Mauvaise carte : " + this.card;
    }

    public Card getCard() {
        return this.card;
    }
}
```

Capture d’une Exception personalisée

```
try{
    Card c = new Card(1,"Pique");
    throw new WrongCardException(c);
} catch (WrongCardException e) {
    System.out.println(e);
    System.out.println(e.getCard());
}
```

Exercices